

Python Environment Setup: Installing PIP & Jupyter

Method 1: Windows Installation Using get-pip.py

Victor Wandera Lumumba, MSc.

Statistician | Data Analyst | ML Expert

Mediacrest Training College

8th Oct 2026

Abstract

This presentation provides a comprehensive, step-by-step guide to setting up a robust Python development environment on Windows, focusing on the manual installation of PIP and the Jupyter ecosystem.

- **Core Objective:** Successfully install and configure the Python Package Installer (PIP) using the official `get-pip.py` script.
- **Extension:** Leverage PIP to install the Jupyter Notebook and JupyterLab interactive computing environments.
- **Verification:** Implement rigorous command-line checks to ensure toolchain integrity and correct PATH configuration.
- **Best Practices:** Address common Windows-specific pitfalls, including environment variable management and permission handling.

Session Objectives

- Understand the role of PIP in the Python ecosystem and why manual installation is sometimes necessary.
- Execute the cURL command to securely download the official `get-pip.py` bootstrap script.
- Run the Python interpreter to execute the installation script and configure core packaging tools.
- Verify the PIP installation using built-in help and versioning commands.
- Install and verify the Jupyter ecosystem (Notebook, JupyterLab, and core components).
- Troubleshoot common Windows errors, such as "PIP is not recognized as an internal or external command."

Part 1: Introduction to PIP

"Understanding the Python Package Manager."

What is PIP?

- **Definition:** PIP stands for "PIP Installs Packages" (a recursive acronym). It is the standard package manager for Python.
- **Primary Function:** It allows you to install, upgrade, and manage third-party libraries and dependencies from the Python Package Index (PyPI).
- **Why Manual Installation?** While modern Python installers (python.org) include PIP by default, manual installation via `get-pip.py` is crucial when:
  - PIP was intentionally omitted during Python installation.
  - The existing PIP installation is corrupted or outdated.
  - Working in restricted or customized enterprise environments.

Prerequisites for Installation

Before You Begin

Ensure the following conditions are met on your Windows machine:

- **Python Installed:** Python must already be installed on the system. Verify with `python -version`.
- **Command Prompt Access:** Ability to open Command Prompt (cmd) or PowerShell. *Running as Administrator is highly recommended to avoid permission errors.*
- **Internet Connection:** Required to download the `get-pip.py` script and later fetch packages from PyPI.
- **cURL Availability:** Windows 10 (build 17063+) and Windows 11 include cURL by default. Older versions may require alternative download methods (e.g., web browser).

Part 2: PIP Installation Steps

"Downloading, Installing, and Verifying."

Step 1: Download get-pip.py

The Bootstrap Script

The `get-pip.py` file is an official, securely maintained Python script provided by the Python Packaging Authority (PyPA) that handles the entire installation process automatically.

Command Prompt Implementation

```
# Navigate to a known directory (e.g., Downloads or C:\)
cd C:\Users\YourUsername\Downloads

# Download the script using cURL
curl https://bootstrap.pypa.io/get-pip.py -o get-pip.py

```

Code Explanation

- `curl`: The command-line tool for transferring data with URLs.
- `-o get-pip.py`: Specifies the output filename, saving the script locally in the current working directory.

Step 2: Installing PIP on Windows

Execution Command

```
# Ensure you are in the same directory as get-pip.py
python get-pip.py

```

What Happens Behind the Scenes?

When executed, this script does more than just install PIP. It performs the following actions:

- **Installs PIP:** The core package manager.
- **Installs Setuptools:** A library that facilitates the packaging and distribution of Python projects.
- **Installs Wheel:** A built-package format that significantly speeds up the installation process by avoiding the need to compile source code.

Note

If you encounter a "Permission Denied" error, close the Command Prompt and reopen it by right-clicking and selecting **"Run as Administrator"**.

Step 3: Verify Installation

Verification Commands

```
# 1. Check if the pip module is accessible via Python
python -m pip help

# 2. Check the specific installed version of PIP
pip --version

```

Expected Output & Explanation

- `python -m pip help`: This is the safest way to invoke PIP. It tells the Python interpreter to run the `pip` module. If this works, PIP is correctly installed.
- `pip -version`: Outputs something like `pip 23.2.1 from C:311-packages(python 3.11)`. This confirms both the version number and the exact installation path, proving Windows can locate the executable.

Part 3: Jupyter Ecosystem Setup

"Empowering Interactive Data Science."

Installing Jupyter Notebook and JupyterLab

Python Implementation

```
# Install the classic Jupyter Notebook interface
pip install notebook

# Install the modern, feature-rich JupyterLab interface
pip install jupyterlab

# (Optional) Install the meta-package that includes both and more
pip install jupyter

```

Component Explanation

- **Jupyter Notebook:** The original, lightweight, document-centric interface ideal for quick scripting and sharing.
- **JupyterLab:** The next-generation user interface. It supports multiple tabs, integrated terminals, code consoles, and advanced file browsing, functioning more like a full IDE.
- **Meta-package (‘jupyter‘):** A convenience package that installs the core Jupyter components, ensuring all

Verifying Jupyter Installation

Verification Command

```
# Check the overall Jupyter environment version and components
jupyter --version

```

Expected Output & Explanation

Running this command will output a list of installed Jupyter components and their respective versions, for example:

- `Selected Jupyter core packages...`
- `IPython: 8.14.0`
- `jupyter_core: 5.3.1`
- `jupyter_server: 2.7.0`
- `notebook: 7.0.0`
- `jupyterlab: 4.0.4`

Note: Seeing these versions confirms that the executables were successfully compiled, placed in the Scripts

Troubleshooting Common Issues

Error: "pip is not recognized as an internal or external command"

Cause: The directory containing `pip.exe` (usually `C:`) is not in the Windows PATH environment variable.

Solutions

1. **Quick Fix:** Always use `python -m pip` instead of just `pip`. This bypasses the PATH requirement by invoking the module directly through the Python interpreter.
2. **Permanent Fix:** Add the Python `Scripts` folder to your Windows Environment Variables:
   - Search Windows for "Environment Variables".
   - Edit the "Path" variable.
   - Add a new entry: `C:3XX`
   - Restart the Command Prompt.

Summary & Best Practices

Key Takeaways

- `get-pip.py` is the most reliable, official method for bootstrapping PIP on Windows.
- Always verify installations using `-version` flags to confirm successful setup.
- JupyterLab is the recommended modern interface for data science workflows, superseding the classic Notebook.

Pro Tips for Maintenance

- **Keep PIP Updated:** Regularly run `python -m pip install -upgrade pip` to ensure you have the latest security patches and features.
- **Use Virtual Environments:** Avoid installing packages globally. Use `python -m venv myenv` to create isolated environments for different projects, preventing dependency conflicts.

Victor Wandera Lumumba
Statistician | Data Analyst | ML Expert
Mediacrest Training College